

CS383 - Software Engineering

Model Driven Engineering

Outline

- Model driven engineering: modeling and metamodeling
- Model management tasks with examples

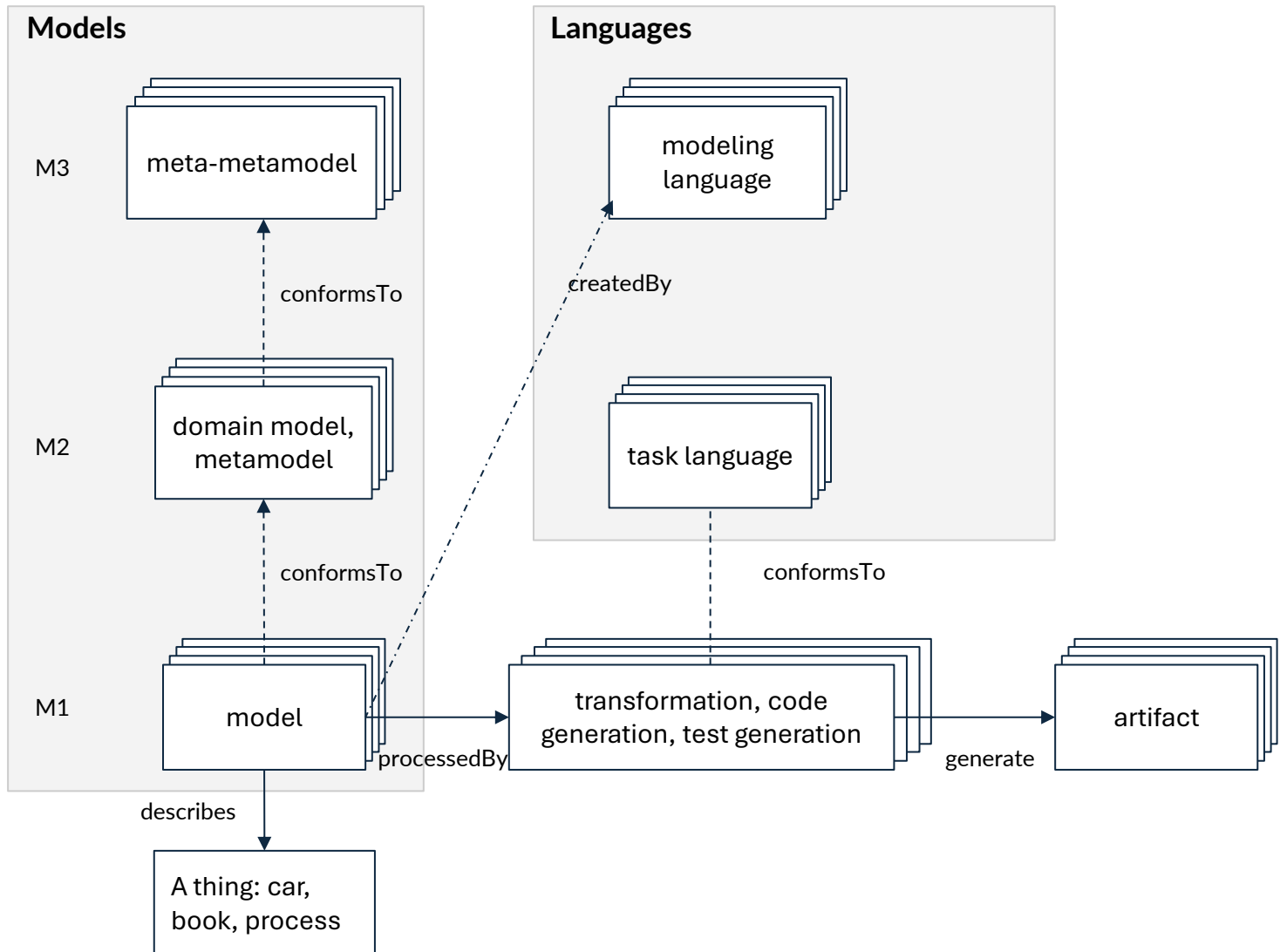
Model Driven Engineering

- It is an approach to software engineering where models are main artifacts
- Models are used for software process, modeling, code generation, testing, evolution, migration and reverse engineering

Model Driven Engineering (cont.d)

- Why MDE?
 - Models are used to deal with complex systems
 - They are abstract and lack of implementation
 - Models are easy to be manipulated and transformed and therefore enable:
 - Understandability: easy to be understood by clients and people involved and requirements checking
 - Increase productivity: with transformation, code correspond to system can be generated
 - More reliability of development: using automated transformation, we have less problems
 - Enable interoperability: using standard like UML, MOF, Ecore, etc that are known by all systems

MDE, model layers and languages



MDE: modeling languages

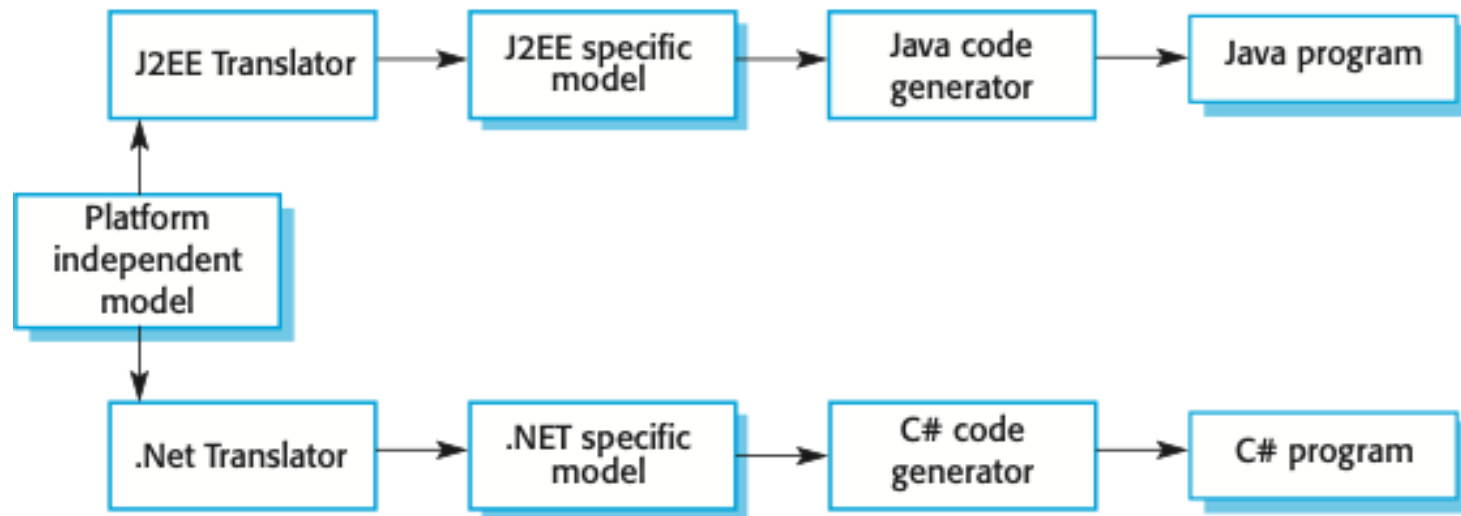
- Modeling languages
 - Provides syntaxes and concepts that can be used to create models
 - Examples of modeling languages
 - UML
 - Ecore: implementation of EMOF that UML conforms to.
 - BPMN (Business Process Model and Notation): use to represents business processes in business process model.
 - Like activity diagram but more tailored to business processes

MDE: Models

- Models are created by modeling language
- They used to represent real object (e.g. Book, Car, Animal, etc)
- A Model in MDE must conform to its modeling language model
 - Why?
 - Easy to be managed and processes
 - Validated and verified
 - Transformed to another type of artifact
- Models in MDE are serialized into XMI (XML Metadata Interchange)

MDE: Models (cont.d)

- Models are transformed from independent model to specific models that works underline systems



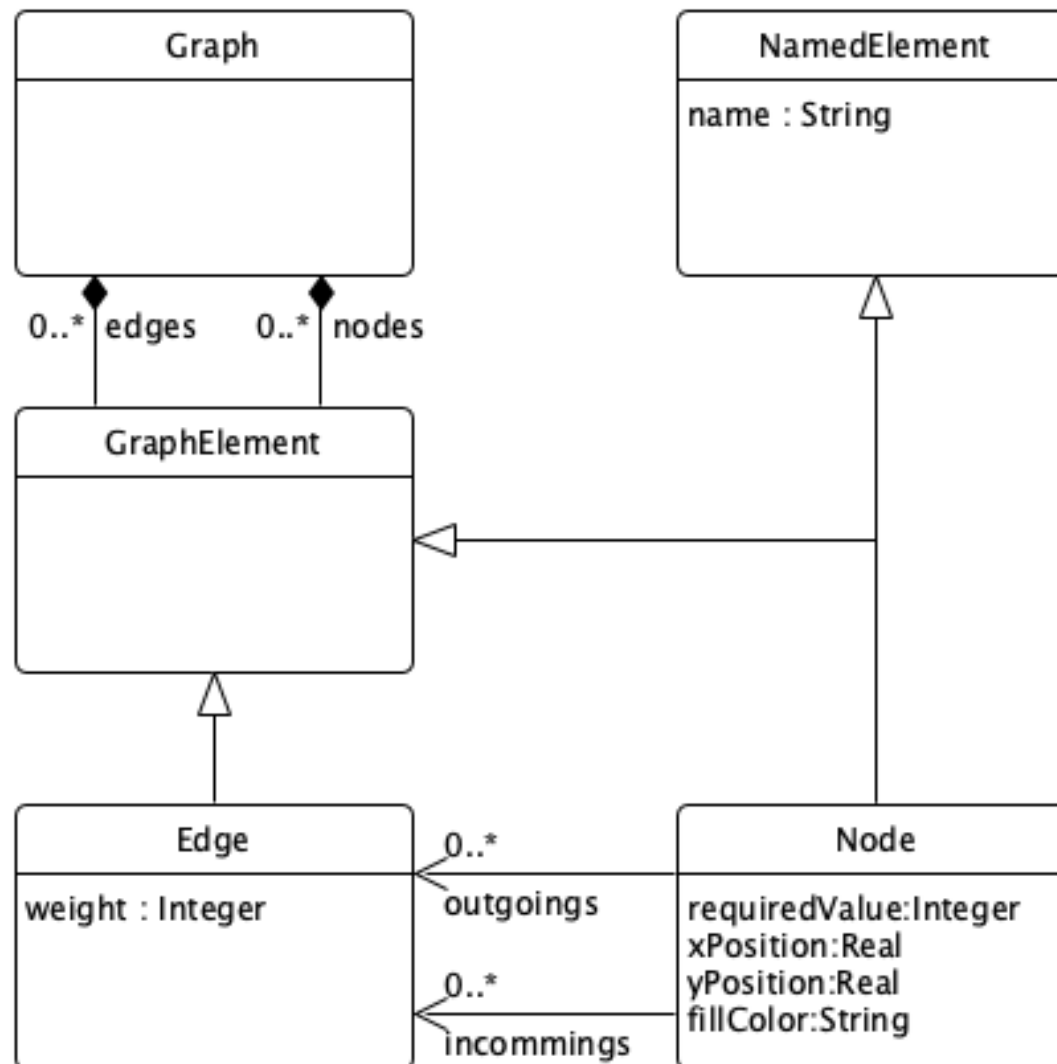
MDE: task management languages

- Models can be managed using task languages
 - Model transformation
 - Create a model from source a model
 - Eg, create Publication model from Book model
 - Model validation
 - Check if a model conforms to its schema (language, constraints, etc)
 - Eg, a book must have at least one chapter and each must have at least 10 pages
 - Model comparison
 - Check the different of two models
 - Eg, does the book name of model A equals to the book name of model B
 - ... and more

MDE task: metamodel creation

- A model must confirm to a metamodel (schema)
- Just like creating a class diagram using UML
- You can create a model graphically or textually
 - Then, you can use auto reverse engineering tools to produce the other artifacts
- Metamodels can be integrated with constraints and invariants
 - Any model generated from the metamodel must be valid after that
 - Eg. all medical staff must have less than year-old issued/renewed licence

MDE task: metamodel graphical example



MDE task: metamodel textual example (in EMFatic)

```
abstract class NamedElement {  
    attr String[1] name;    // attribute single variable  
}  
  
abstract class GraphElement{ }  
  
class Graph {  
    val GraphElement[0..*] nodes; // contained collection variable  
    val GraphElement[0..*] edges;  
}  
  
class Node extends NamedElement, GraphElement {  
    attr int requiredValue;        // attribute single variable  
    ref Edge[0..*]#sourceNode outgoing; // uncontained collection variable  
    ref Edge[0..*]#targetNode incommings;  
    attr float xPosition;  
    attr float yPosition;  
    attr String fillColor;  
}  
  
class Edge extends GraphElement{  
    attr int weight;  
    ref Node[1]#outgoings sourceNode;  
    ref Node[1]#incommings targetNode;  
}
```

MDE task: model generation

- Once we created the domain model (metamodel), we can generate an object model that confirms to it
 - In other words, instance of metamodel
- Again, object models can be generated textually or graphically
 - Graphically drawbacks
 - Time consuming, not flexible
 - Textual is a solution as it enables automation
 - Any changes to metamodel can be propagated into models easily
- Model generators can be written using general-purpose languages like Java and C++. However, there are dedicate languages for that like Epsilon object language

MDE task: model generation example (EOL)

```
var n1:Node = GraphMM!Node.createInstance(); // create node 1
n1.name = "Buraydah";
n1.requiredValue = 5;
n1.xPosition = 41515;
n1.yPosition = 51277;
n1.fillColor = "Red";

var n2:Node = GraphMM!Node.createInstance(); // create node 2
n2.name = "Riyadh";
n2.requiredValue = 15;
n2.xPosition = 81225;
n2.yPosition = 66143;
n2.fillColor = "Blue";

var e1:Edge = GraphMM!Edge.createInstance(); // create edge between n1 and n2
e1.weight = 10;
e1.sourceNode = n1;
e1.targetNode = n2;

var g:Graph = GraphMM!Graph.createInstance(); // create a graph instance
g.nodes.add(n1); // add nodes to graph, note that 'nodes' is a collection
g.nodes.add(n2);

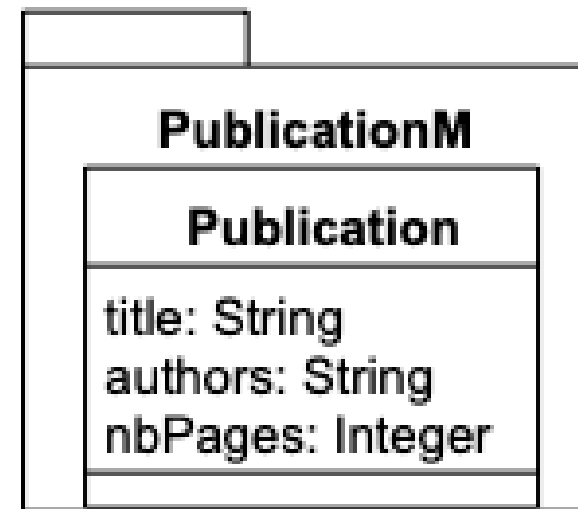
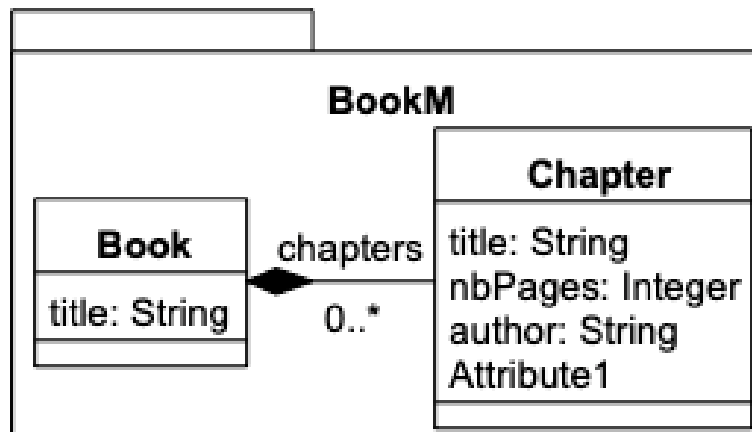
g.edges.add(e1); // add edges to graph
// now the graph 'g' has two nodes and an edge that connects the nodes
```

MDE task: model transformation

- The main task in MDE is model transformation
 - One or several models are used to generate other models
 - Eg. use classical car model to generate a new autonomous car model with more features and safety constraints
 - Eg. migrate an old model system to a new system with new infrastructural and architectural changes
- Basically, you query the source models and get necessary information (mapping) to generate target models
 - Metamodels of both source and target must be used for navigation and creation of objects in the output models

MDE task: model transformation example

- Assume that we have these metamodels



- Also assume that we have models (objects) that conform to BookM
- We want to use a model that conforms to BookM to generate (transform) to a new model that conforms to PublicationM

MDE task: model transformation example

```
-- a transformation program written using Atlas Transformation  
-- Language (ATL)
```

```
rule Book2Publication {  
  from b : BookM!Book  
  to out : PublicationM!Publication (  
    title <- b.title,  
    authors <- b.getAuthors(), -- calling a helping method  
    nbPages <- b.getSumPages() -- calling a helping method  
  )  
}
```

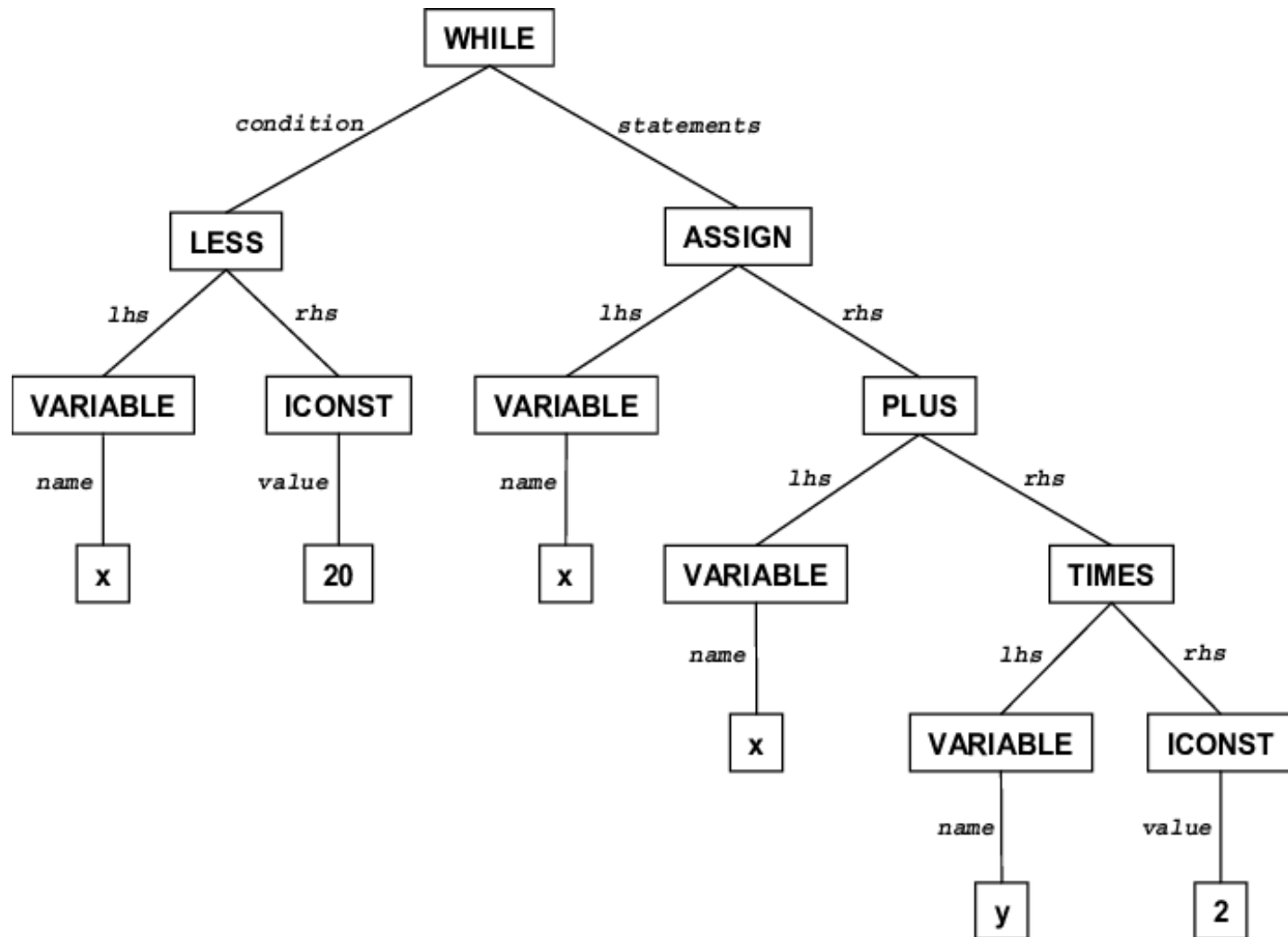
MDE task: model to text transformation

- Models are universal and they are system-independent.
- Model2text takes a model as an input and create code of system language
 - Eg, take a model of Book and generate a program that create/update/delete a book to/from DB
 - This can be according to target system, for example PHP with SQL capability, or Django with SQL capability, etc.
 - We can specify several model2text code to different system

MDE task: text to model transformation

- text2model: takes a text as an input and generate a model
 - To be able to do that, we need static analyser or parsers
 - Eg, Parse java code and generate Abstract Syntax Tree
 - Eg, Parse a text weather text and numbers and generate a graphical representation

MDE task: text to model transformation



MDE task: model validation

- Metamodel can contain constraints and a set of invariant (conditions)
- These are called conformance conditions
- A model must conform to its metamodel
- Therefore, it is important to check models from time to time if they are still valid
 - Valid according to its constraints
- Eg. a book must have a name and at least 10 pages, otherwise it is not a valid book
- Eg. a car must conform to the local government's standards that are integrated into the metamodel

MDE task: model comparison

- Because models are manipulated by management tasks
 - It is a requirement to check if the out models are generated with no errors
 - check the model and compare it with an expected output (oracle)
 - Any data can be changed according to three actions: add, delete, replace
 - Comparison relies on these actions
 - A comparison can be very good for merging models
 - E.g want to see what are the missing data from a model and then amend it with merging information from a sample model